

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра ИС

ОТЧЕТ
по практической работе №1
по дисциплине «Методы и средства проектирования информационных
систем»
Тема: Приемы моделирования классификатора изделий

Студентки гр. 3395	_____	Аришина А.А.
	_____	Мансурова С.Н.
Студентка гр. 3372	_____	Пискунова З. А.
Преподаватель	_____	Дубенецкий В.А.

Санкт-Петербург
2026

ЗАДАНИЕ НА ПРАКТИЧЕСКУЮ РАБОТУ

Студентка гр. 3395 Аришина А. А.

Студентка гр. 3395 Мансурова С. Н.

Студентка гр. 3372 Пискунова З. А.

Тема работы: приемы моделирования классификатора изделий на примере платформы для каршеринга

Исходные данные:

Создать классификатор платформы для каршеринга, хранящийся в базе данных и сервер с API, обеспечивающий доступ и работу с классификатором.

Содержание пояснительной записки:

«Аннотация», «Содержание», «Введение», «Анализ предметной области», «Разработка функциональных требований», «Разработка проектной модели», «Разработка модели хранения», «Разработка базы данных», «Разработка серверной части», «Заключение», «Список использованных источников», «Приложение».

Предполагаемый объем пояснительной записки:

Не менее 20 страниц.

Дата выдачи задания: 19.02.2026

Дата сдачи отчета: 05.04.2026

Дата защиты работы: 06.04.2026

Студентки гр. 3395

Аришина А.А.

Мансурова С.Н.

Студентка гр. 3372

Пискунова З. А.

Преподаватель

Дубенецкий В.А.

АННОТАЦИЯ

В данной работе разработан прототип подсистемы управления иерархическим классификатором автомобилей для платформы каршеринга. Проведен анализ предметной области, сформулированы функциональные требования и построена модель вариантов использования (Use Case). Спроектирована реляционная структура данных и реализована база данных на SQLite с использованием ORM SQLAlchemy. Серверная часть приложения разработана на Python (Flask) и предоставляет REST API для управления древовидной структурой классов и экземплярами автомобилей. Реализованы механизмы обеспечения целостности данных, включая проверку на циклические зависимости при изменении иерархии. Выполнено тестирование всех функций системы, подтвердившее корректность работы алгоритмов добавления, перемещения и удаления узлов дерева без нарушения логической структуры.

SUMMARY

This work presents the development of a prototype subsystem for managing a hierarchical vehicle classifier for a car-sharing platform. The subject domain was analyzed, functional requirements were formulated, and a Use Case model was constructed. A relational data structure was designed, and a database was implemented using SQLite with the SQLAlchemy ORM. The server-side application was developed in Python (Flask), providing a REST API for managing the tree-like class hierarchy and vehicle instances. Data integrity mechanisms were implemented, including cycle detection during hierarchy modifications. System testing confirmed the correct operation of algorithms for adding, moving, and deleting tree nodes without compromising the logical structure.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
2. РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ	9
3. РАЗРАБОТКА ПРОЕКТНОЙ МОДЕЛИ	12
4. РАЗРАБОТКА МОДЕЛИ ХРАНЕНИЯ	14
5. РАЗРАБОТКА БАЗЫ ДАННЫХ	16
6. РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ	20
ЗАКЛЮЧЕНИЕ	35
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	36
ПРИЛОЖЕНИЕ А. ЛИСТИНГ	37

ВВЕДЕНИЕ

Целью данной работы является разработка прототипа подсистемы управления иерархическим классификатором автомобилей для платформы каршеринга. Актуальность задачи обусловлена необходимостью создания гибкой системы каталогизации, позволяющей администраторам самостоятельно изменять структуру справочника без участия разработчиков, а пользователям – эффективно искать данные о транспортном парке.

Для достижения цели решены следующие задачи:

- Проведен анализ предметной области и сформулированы функциональные требования.
- Разработана модель вариантов использования (Use Case) и проектная модель системы (диаграмма классов UML).
- Спроектирована реляционная модель данных и реализована база данных на SQLite с использованием ORM SQLAlchemy.
- Создан сервер на Python (Flask), предоставляющий REST API для управления иерархией классов и экземплярами автомобилей.
- Выполнено тестирование системы, включая проверку целостности данных и защиту от циклических зависимостей.

1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

Объектом автоматизации является каталог автомобилей каршеринговой платформы. В распоряжении компании находится большой парк транспортных средств, который включает легковые и грузовые автомобили.

Цель разработки – создание единого иерархического классификатора автомобилей в виде древовидной структуры.

Основные возможности классификатора:

- Логическая группировка: Автомобили объединяются в категории и подкатегории.
- Удобный поиск: при выборе категории пользователь сразу видит все модели, входящие в неё.
- Гибкость настройки: администратор может самостоятельно изменять структуру каталога – добавлять, удалять и перемещать категории без изменения кода.
- Базовая единица: для всех классов – «штука».

Структура классификатора реализована по типу «конструктора». Администратор самостоятельно строит иерархию, устанавливая родительско-дочерние связи. Система контролирует целостность данных и предотвращает некорректные действия (циклические ссылки).

В качестве примера была выбрана платформа CityDrive. Это современная платформа каршеринга, которая реализует поминутную или почасовую аренду легковых автомобилей через мобильное приложение. Автомобили в каталоге различаются по множеству взаимосвязанных признаков, которые образуют иерархическую структуру классификатора. Для нашей системы выбрана следующая структура, изображённая на рисунке 1, которая включает в себя возможность аренды и легковых, и грузовых автомобилей.

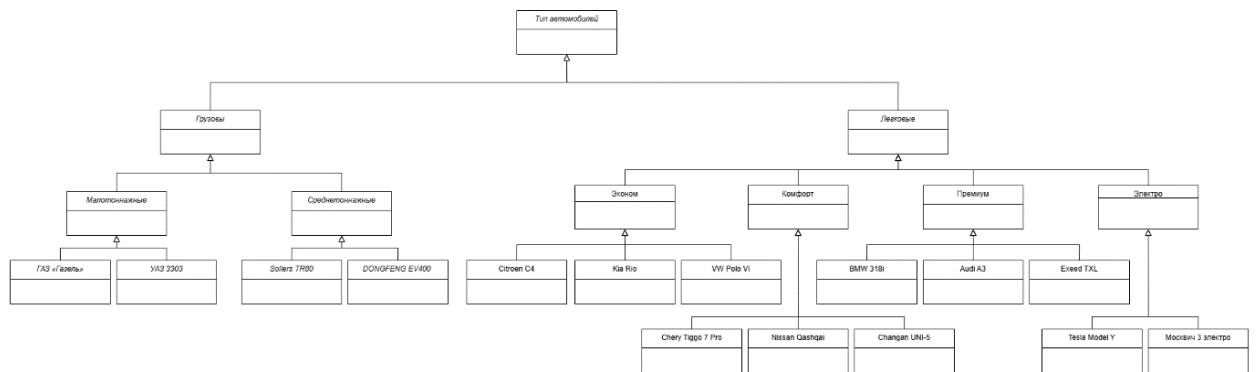


Рисунок 1 – Структура данных

Получаем следующую иерархию классов:

- Тип автомобилей (корень дерева)
 - Грузовые
 - Малотоннажные
 - ГАЗ «Газель» (терминальный класс)
 - УАЗ 3303 (терминальный класс)
 - Среднетоннажные
 - Sollers TR80 (терминальный класс)
 - DONGFENG EV400 (терминальный класс)
 - Легковые
 - Эконом
 - Citroen C4 (терминальный класс)
 - Kia Rio (терминальный класс)
 - VW Polo VI (терминальный класс)
 - Комфорт
 - Chery Tiggo 7 Pro (терминальный класс)
 - Nissan Qashqai (терминальный класс)
 - Changan UNI-5 (терминальный класс)
 - Премиум
 - Audi A3 (терминальный класс)
 - BMW 318i (терминальный класс)

- Exeed TXL (терминальный класс)
- Электро
 - Tesla Model Y (терминальный класс)
 - Москвич 3 электро (терминальный класс)

Корнем данной иерархической структуры является общий класс «Тип автомобилей», который объединяет все последующие категории. Промежуточными узлами выступают классы «Грузовые» и «Легковые», а также их подразделы, такие как «Малотоннажные», «Среднетоннажные», «Эконом», «Комфорт», «Премиум» и «Электро». Эти промежуточные уровни служат для логической группировки конкретных моделей по назначению и характеристикам, не являясь конечными элементами системы. Терминальными классами, или листьями дерева, являются конкретные марки и модели автомобилей, например, ГАЗ «Газель», Kia Rio или Tesla Model Y. Именно на уровне терминальных классов иерархия завершается, так как они не имеют дальнейших вложенных подкатегорий.

2. РАЗРАБОТКА ФУНКЦИОНАЛЬНЫХ ТРЕБОВАНИЙ

На основе проведенного анализа предметной области (Раздел 1) были сформированы функциональные требования к подсистеме управления классификатором автомобилей. Для формализации этих требований и визуализации взаимодействия пользователей с системой была построена модель вариантов использования (Use Case Model) на языке UML. Особое внимание уделено требованиям к гибкости структуры, заявленным в задании: система должна позволять администратору изменять иерархию (добавлять, перемещать, удалять узлы) без вмешательства разработчиков, а также гарантировать целостность данных (отсутствие циклов, уникальность кодов) при любых операциях. Ниже приведено описание действующих лиц и основных сценариев использования, которые визуализированы на диаграмме вариантов использования (Рисунок 2).

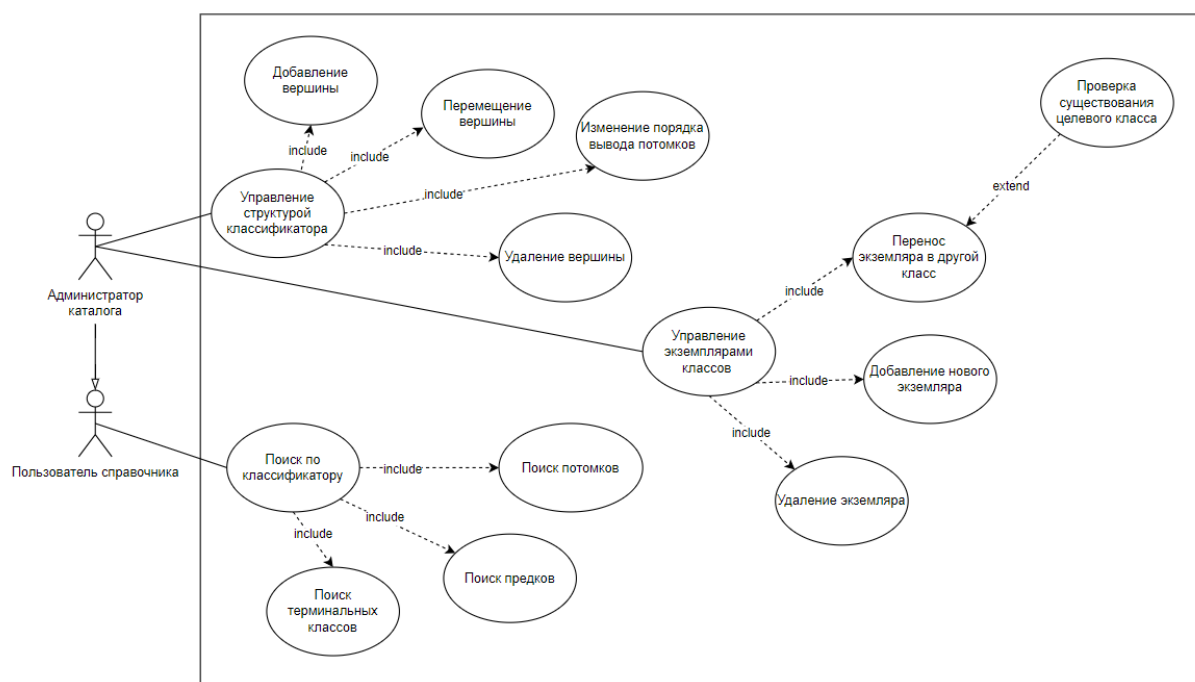


Рисунок 2 – Диаграмма Use Case

Взаимодействие с системой разделено на два основных направления: администрирование структуры справочника и эксплуатация (поиск и просмотр данных).

Основными действующими лицами (actor) являются:

1. Пользователь справочника – сотрудник, использующий классификатор только в режиме просмотра и поиска. Не имеет прав на изменение структуры.

2. Администратор каталога – сотрудник, обладающий правами на изменение структуры классификатора и управление экземплярами классов. Выполняет операции по добавлению, удалению, перемещению и редактированию классов и экземпляров, а также имеет возможность поиска по классификатору (как пользователь справочника).

Основные сценарии использования.

Поисковые функции:

- Поиск потомков: отображение всех подкатегорий и моделей, входящих в выбранный тип или категорию автомобиля.
- Поиск предков: определение цепочки категорий, в которую входит выбранная модель (например, к какому типу и классу относится автомобиль).
- Поиск терминальных классов: получение списка всех конечных моделей автомобилей, к которым могут быть привязаны реальные машины в парке.

Управление структурой:

- Добавление вершины: создание новой категории (промежуточный класс) или новой модели автомобиля (терминальный класс) в выбранной ветке классификатора.
- Перемещение вершины: изменение принадлежности категории или модели, перенос в другую ветку классификатора с автоматической проверкой на отсутствие заикливания.
- Изменение порядка вывода потомков: настройка последовательности отображения категорий или моделей внутри выбранного раздела.

- Удаление вершины: исключение категории или модели из классификатора с контролем наличия вложенных элементов и привязанных автомобилей в парке.

Управление экземплярами классов:

- Добавление нового экземпляра: регистрация конкретного автомобиля в системе.

- Удаление экземпляра: удаление конкретного автомобиля из базы данных по его уникальному идентификатору.

- Перенос экземпляра в другой класс: обновление принадлежности автомобиля к другому классу.

3. РАЗРАБОТКА ПРОЕКТНОЙ МОДЕЛИ

Для рассматриваемой системы была составлена диаграмма классов, представленная на рисунке 3. Она отображает ключевые атрибуты и методы классов, а также отношения между ними.

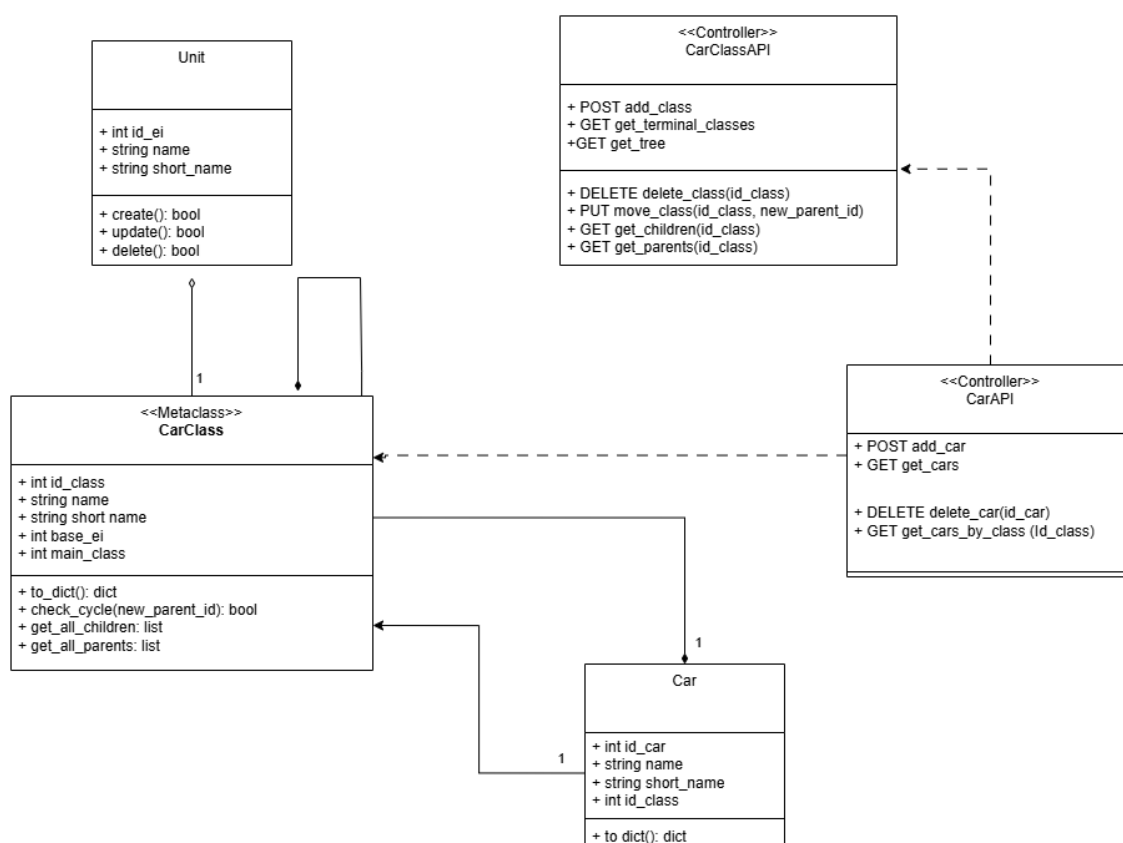


Рисунок 3 – Диаграмма классов

Класс **CarClass** является метаклассом и представляет узел дерева категорий автомобилей. Он используется для организации иерархической структуры классификатора, позволяя группировать автомобили по типу (легковые/грузовые), классу (эконом, комфорт и т.д.) и модели. Этот класс имеет рефлексивную связь с самим собой: один родительский узел может содержать несколько дочерних узлов, формируя древовидную структуру. Также класс связан с таблицей единиц измерения (**Unit**) через поле `base_ei`.

Класс Car описывает конкретные автомобили в парке каршеринга. Включает такие атрибуты, как госномер (short_name), описание (name) и ссылку на класс (id_class). Каждый автомобиль принадлежит ровно одному узлу классификатора, поэтому связан с CarClass ассоциацией «многие к одному».

Класс Unit хранит справочник единиц измерения. Используется для указания базовой единицы измерения для каждого класса автомобилей. Связан с CarClass через внешний ключ base_ei.

Класс ClassifierAPI представляет собой контроллер (или фасад) для управления структурой классификатора. Он инкапсулирует логику взаимодействия с клиентом через REST API. Включает методы для обработки HTTP-запросов: POST add_class, DELETE delete_class, PUT move_class, а также методы чтения: GET get_tree, GET get_children и GET get_parents, GET get_terminal_classes. Этот класс зависит от моделей CarClass и Unit для выполнения операций с данными.

Класс CarAPI представляет собой контроллер для управления экземплярами автомобилей. Он обрабатывает запросы, связанные с конкретными машинами. Включает методы: POST add_car, DELETE delete_car, PUT update_car, а также методы чтения: GET get_cars и GET get_cars_by_class. Этот класс зависит от моделей Car и CarClass для обеспечения целостности связей.

Все классы реализованы в виде моделей SQLAlchemy и маршрутов Flask. Диаграмма отражает разделение ответственности: модели (CarClass, Car, Unit) отвечают за хранение и бизнес-логику данных, а контроллеры (ClassifierAPI, CarAPI) – за взаимодействие с внешним миром через HTTP-протокол.

4. РАЗРАБОТКА МОДЕЛИ ХРАНЕНИЯ

Для хранения информации о классификаторе автомобилей используется реляционная модель данных (Рисунок 4), которая включает в себя три основные таблицы: `car_classes`, `cars` и `ei`.

В таблице `car_classes` каждой записи присвоен уникальный идентификатор `id_class`, который является первичным ключом. Для реализации иерархической структуры каждый узел может ссылаться на родителя через поле `main_class`, которое является внешним ключом, ссылающимся на `id_class` той же таблицы. Это позволяет строить дерево классификации, где один узел может иметь несколько дочерних, а каждый дочерний не более одного родителя. Кроме того, таблица содержит поля `name` и `short_name` для обозначения класса, а также поле `base_ei` для указания единицы измерения.

Таблица `cars` хранит информацию о конкретных автомобилях и имеет свой первичный ключ `id_car`. Каждая запись связывается с определённым узлом классификатора через поле `id_class`, которое является внешним ключом и обеспечивает связь одного узла с множеством автомобилей. Дополнительные атрибуты таблицы включают `short_name` для уникального обозначения автомобиля и `name` для описания автомобиля.

Таблица `ei` хранит справочник единиц измерения и содержит первичный ключ `id_ei`, а также поля `short_name` и `name` для указания кода и наименования единицы измерения. Данная таблица связана с таблицей `car_classes` через поле `base_ei`, позволяя задавать единицу измерения для классов.

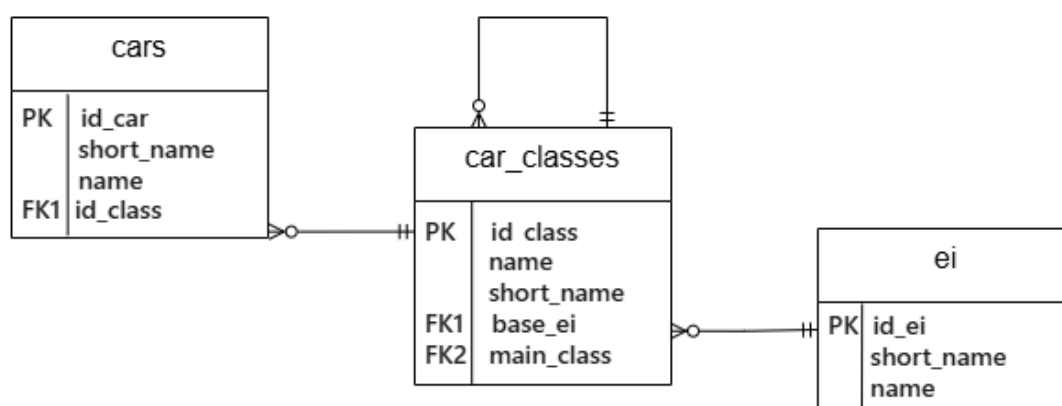


Рисунок 4 – ER-диаграмма

Модель разработана с учётом принципов нормализации, что минимизирует избыточность данных и упрощает поддержку актуальности информации. Взаимодействие между таблицами отражено на ER-диаграмме, которая служит основой для последующей физической реализации базы данных в СУБД SQLite с использованием ORM SQLAlchemy.

5. РАЗРАБОТКА БАЗЫ ДАННЫХ

Для работы была выбрана встроенная реляционная СУБД SQLite, так как она не требует отдельного серверного процесса, обеспечивает надёжное хранение структурированных данных и полностью совместима с ORM-фреймворком SQLAlchemy. Такое решение упрощает развёртывание и отладку системы, позволяя хранить иерархический классификатор в одном файле без дополнительной настройки окружения.

Инициализация базы данных осуществляется автоматически при запуске Flask-приложения с помощью метода `db.create_all()`, который формирует структуру таблиц на основе объявленных моделей (`CarClass`, `Car`, `Unit`). В случае отсутствия записей система выполняет проверку и добавляет корневой узел классификатора и базовую единицу измерения, что гарантирует готовность рабочей среды сразу после старта сервера.

В нашей реализации нет хранимых процедур или функций базы данных. Весь код написан на Python с использованием ORM SQLAlchemy. Это означает, что логика выполняется на стороне приложения (бэкенда), а база данных используется только для хранения данных и выполнения базовых CRUD-операций (`SELECT`, `INSERT`, `UPDATE`, `DELETE`). Все функции с данными выполняются внутри классов и маршрутов API. Основные методы обработки данных на стороне сервера и маршруты взаимодействия с клиентом представлены в таблицах 1 и 2.

Таблица 1 – Логические функции моделей

Название функции / Метода	Назначение	Вид	Выход
<code>CarClass.to_dict</code>	Преобразование объекта класса автомобиля в словарь.	<code>self</code>	dict: { <code>id_class</code> , <code>name</code> , <code>short_name</code> , <code>base_ei</code> , <code>main_class</code> }

Продолжение таблицы 1 – Логические функции моделей

Название функции / Метода	Назначение	Вид	Выход
CarClass.check_cycle	Проверка иерархии на наличие циклов при изменении родителя	self, new_parent_id	bool: True, False
CarClass.get_all_children	Рекурсивный поиск всех дочерних классов текущего класса.	self	list: Список объектов CarClass
CarClass.get_all_parents	Поиск всей цепочки родительских классов текущего класса.	self	list: Список объектов CarClass
Car.to_dict	Преобразование объекта автомобиля в словарь.	self	dict: {id_car, short_name, name, id_class}
Unit.to_dict	Преобразование единицы измерения в словарь.	self	dict: {id_ei, short_name, name}

Таблица 2 – Функции API

Название функции	Назначение	Вид	Выход
add_class (POST)	Добавление нового класса	name, short_name, base_ei, main_class	JSON с id нового класса
delete_class (DELETE)	Удаление класса с переносом детей	id_class	Статус удаления, сообщение о переносе
move_class (PUT)	Смена родителя (перемещение)	id_class, new_parent_id	Статус moved или ошибка цикла
get_children (GET)	Получение всех потомков	id_class	Список дочерних классов
get_parents (GET)	Получение цепочки предков	id_class	Список родительских классов

Название функции	Назначение	Вид	Выход
get_terminal_classes (GET)	Получение всех терминальных (листовых) классов	all_classes	Список терминальных классов
get_tree (GET)	Получение полной структуры дерева классификатора	—	Дерево классов в формате JSON
add_car (POST)	Добавление автомобиля	short_name, name, id_class	JSON с id авто
delete_car (DELETE)	Удаление конкретного автомобиля	id_car	Статус deleted
update_car (PUT)	Обновление данных автомобиля (класс/описание)	id_car, id_class, name	Обновлённый объект автомобиля и статус
get_cars (GET)	Получение списка всех автомобилей	—	Массив всех автомобилей
get_cars_by_class (GET)	Машины конкретного класса и его подклассов	id_class	Список автомобилей

Представленные функции демонстрируют, что вся бизнес-логика, включая валидацию иерархии, рекурсивный обход дерева и управление связями, инкапсулирована в методах моделей и обработчиках маршрутов. Такой подход обеспечивает высокую читаемость кода, упрощает тестирование.

При запуске приложения у нас создалась БД. В дальнейшем мы её заполним тестовыми данными. Открываем БД в СУБД SQLite схема БД представлена на рисунке 5.

Database Structure		
Создать таблицу Создать индекс Печать Обновить		
Имя	Тип	Схема
Таблицы (3)		
car_classes	CREATE TABLE car_classes (id_class INTEGER NOT NULL, name VARCHAR(100) NOT NULL, short_name VARCHAR(50), base_ei INTEGER, m	
id_class	INTEGER	"id_class" INTEGER NOT NULL
name	VARCHAR(100)	"name" VARCHAR(100) NOT NULL
short_name	VARCHAR(50)	"short_name" VARCHAR(50)
base_ei	INTEGER	"base_ei" INTEGER
main_class	INTEGER	"main_class" INTEGER
cars	CREATE TABLE cars (id_car INTEGER NOT NULL, short_name VARCHAR(20) NOT NULL, name VARCHAR(100), id_class INTEGER NOT NULL,	
id_car	INTEGER	"id_car" INTEGER NOT NULL
short_name	VARCHAR(20)	"short_name" VARCHAR(20) NOT NULL
name	VARCHAR(100)	"name" VARCHAR(100)
id_class	INTEGER	"id_class" INTEGER NOT NULL
ei	CREATE TABLE ei (id_ei INTEGER NOT NULL, short_name VARCHAR(10) NOT NULL, name VARCHAR(50) NOT NULL, PRIMARY KEY (id_ei), U	
id_ei	INTEGER	"id_ei" INTEGER NOT NULL
short_name	VARCHAR(10)	"short_name" VARCHAR(10) NOT NULL
name	VARCHAR(50)	"name" VARCHAR(50) NOT NULL
Индексы (0)		
Представления (0)		
Триггеры (0)		

Рисунок 5 – Схема БД

Функции, представленные ранее в таблицах 1 и 2, будут далее рассмотрены в ходе тестирования работы сервера. Что позволит подтвердить их работоспособность, а также продемонстрировать примеры выполнения операций.

6. РАЗРАБОТКА СЕРВЕРНОЙ ЧАСТИ

Для реализации серверной части приложения был выбран микрофреймворк Flask на языке программирования Python. Данное решение было выбрано благодаря своей легковесности, модульности и удобству интеграции с ORM SQLAlchemy, что позволило эффективно реализовать бэкенд для работы с иерархической структурой данных без необходимости написания сложных SQL-запросов вручную.

Взаимодействие с клиентской частью организовано через REST API. Сервер принимает HTTP-запросы, обрабатывает их, выполняя бизнес-логику и запросы к базе данных, и возвращает ответы в формате JSON. Для обеспечения возможности отправки запросов из веб-браузера (cross-origin requests) используется расширение Flask-CORS.

Маршрутизация запросов реализована через декораторы `@app.route`. Все эндпоинты сгруппированы по префиксам `/api/class` для работы с категориями классификатора (добавление, удаление, перемещение, получение дерева) и `/api/car` для управления конкретными автомобилями.

При обработке запросов сервер выполняет валидацию входных данных и проверку бизнес-правил.

Реализованные методы HTTP и соответствующие им URL-адреса для доступа к функционалу системы представлены в таблице 3.

Таблица 3 – Реализованные методы HTTP

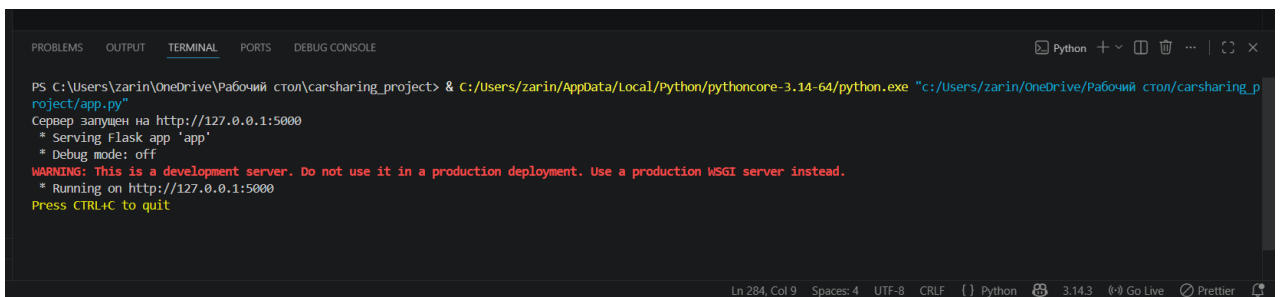
Метод	URL	Описание
POST	<code>http://127.0.0.1:5000/api/class/add</code>	Создать новый узел классификатора
DELETE	<code>http://127.0.0.1:5000/api/class/<id></code>	Удалить узел (с каскадным переносом)
PUT	<code>http://127.0.0.1:5000/api/class/<id>/move</code>	Переместить узел к новому родителю. Внутри данного метода происходит проверка на циклы

Продолжение таблицы 3 – Реализованные методы HTTP

Метод	URL	Описание
GET	http://127.0.0.1:5000/api/class/<id>/children	Получить всех потомков узла
GET	http://127.0.0.1:5000/api/class/<id>/parents	Получить всех предков узла
GET	http://127.0.0.1:5000/api/class/tree	Получить полное дерево классификатора
POST	http://127.0.0.1:5000/api/car/add	Добавить экземпляра
GET	http://127.0.0.1:5000/api/cars/<id_class>	Получить экземпляры по классу
GET	http://127.0.0.1:5000/api/cars	Получение всех экземпляров
DELETE	http://127.0.0.1:5000/api/car/<id>	Удаление экземпляра класса
PUT	http://127.0.0.1:5000/api/car/<id>	Перемещение экземпляра класса
GET	http://127.0.0.1:5000/api/class/terminal	Поиск всех терминальных классов

Серверное приложение запускается на локальном хосте (127.0.0.1) на порту 5000. В рабочей конфигурации режим отладки (debug) отключен для обеспечения стабильности работы системы.

Проведём тестирование разработанной системы. Для начала осуществим запуск серверной части приложения (Рисунок 6).



```

PS C:\Users\zarin\OneDrive\Рабочий стол\carsharing_project> & C:/Users/zarin/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/zarin/OneDrive/Рабочий стол/carsharing_p
project/app.py"
Сервер запущен на http://127.0.0.1:5000
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
  
```

Рисунок 6 – Запуск сервера

Все данные в БД были добавлены напрямую через запросы HTTP. На рисунке 7 представлен запрос POST на добавление нового класса «Эконом». Все остальные классы были добавлены идентично.

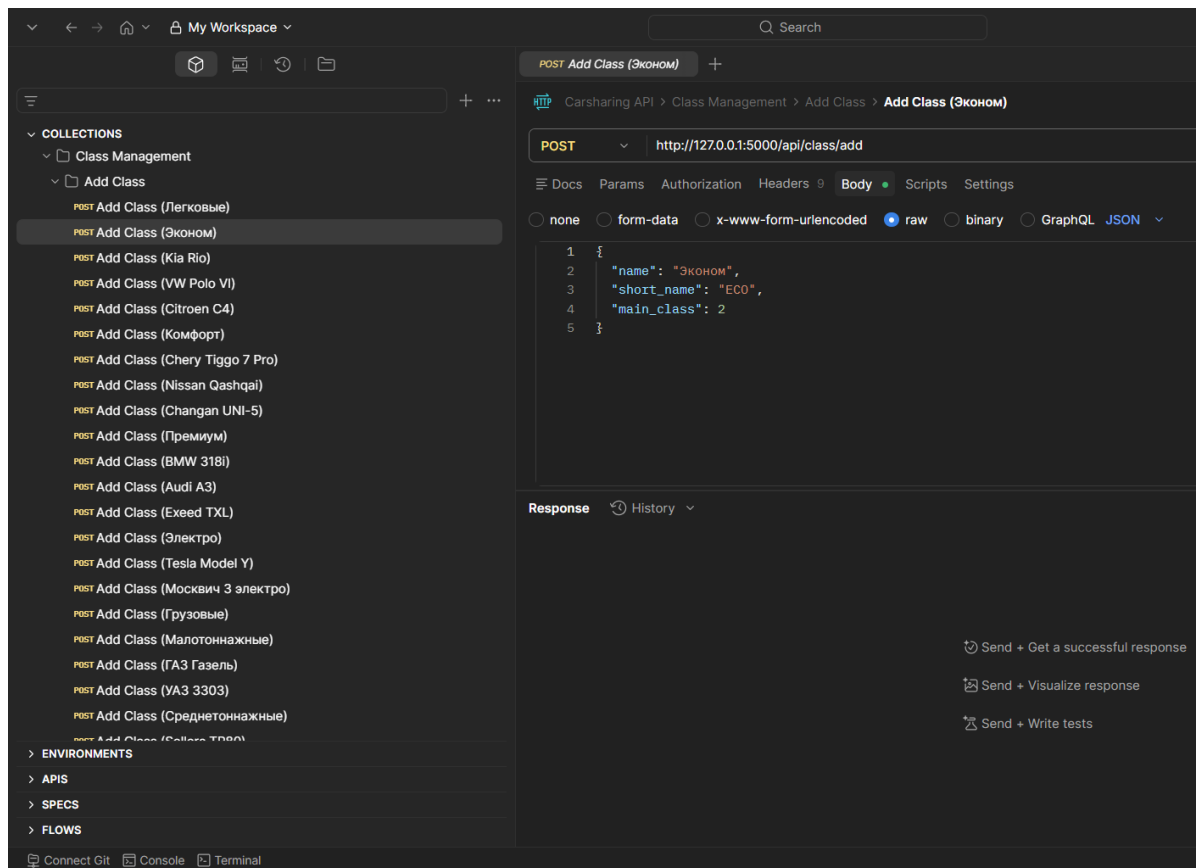


Рисунок 7 – Добавление класса "Эконом"

После добавления всех классов проверяем результат в БД. На рисунке 8 можем увидится, что все классы добавились корректно.

Таблица: car_classes					
	id_class	name	short_name	base_el	main_class
	Фильтр	Фильтр	Фильтр	Фильтр	Фильтр
1	1	Каталог Каршеринга	ROOT	NULL	NULL
2	3	Эконом	ECO	NULL	25
3	5	VW Polo VI	POLO	1	3
4	6	Citroen C4	C4	1	3
5	7	Комфорт	COMFORT	NULL	25
6	8	Chery Tiggo 7 Pro	TIGGO	1	7
7	9	Nissan Qashqai	QASHQAI	1	7
8	10	Changan UNI-5	UNI5	1	7
9	11	Премиум	PREMIUM	NULL	25
10	12	BMW 318i	BMW318	1	11
11	13	Audi A3	AUDI_A3	1	11
12	14	Exeed TXL	EXEED	1	11
13	15	Электро	ELECTRO	NULL	25
14	16	Tesla Model Y	TESLA_Y	1	15
15	18	Грузовые автомобили	CAR_TRUCK	NULL	1
16	19	Малотоннажные	LIGHT_TRUCK	NULL	18
17	20	ГАЗ Газель	GAZELLE	1	19
18	21	УАЗ 3303	UAZ3303	1	19
19	22	Среднетоннажные	MEDIUM_TRUCK	NULL	18
20	23	Sollers TR80	SOLLERS	1	22
21	24	DONGFENG EV400	DF_EV400	1	22
22	25	Легковые автомобили	CAR_PASSENGER	NULL	1
23	26	Kia Rio	RIO	1	3

Рисунок 8 – Таблица car_classes

На рисунке 9 представлен запрос POST на добавление экземпляра класса машина «Kio Rio белый». Все остальные машины были добавлены идентично.

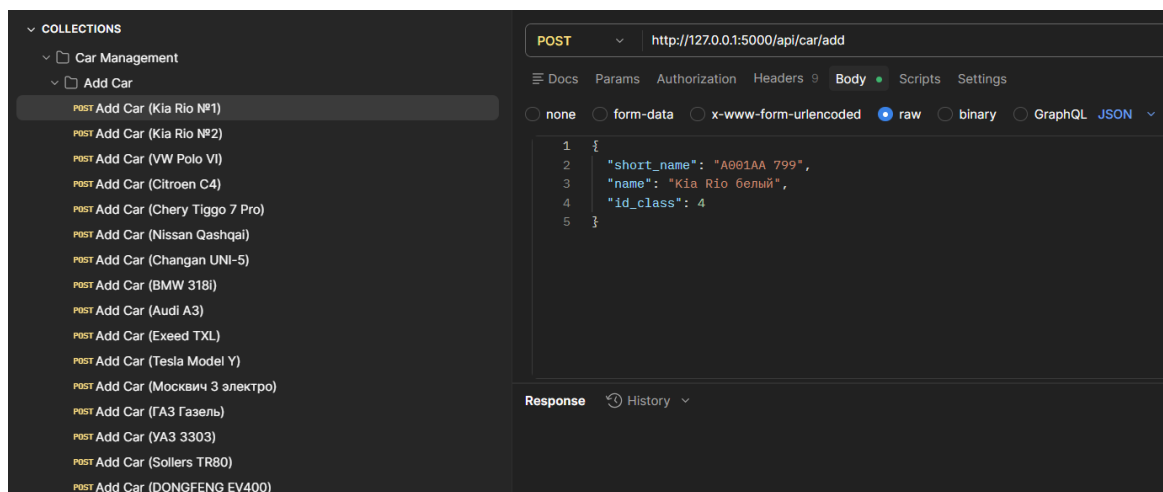


Рисунок 9 – Добавление экземпляра класса

После добавления всех экземпляров класса просматриваем результат запросов в БД. На рисунке 10 можем видеть, что все экземпляры класса добавились корректно.

Таблица: cars

	<u>id_car</u>	short_name	name	id_class
	Филь...	Фильтр	Фильтр	Фильтр
1	1	A001AA 799	Kia Rio белый	26
2	2	B002BB 799	Kia Rio черный	26
3	3	C003CC 799	VW Polo VI серебристый	5
4	4	D004DD 799	Citroen C4 красный	6
5	5	E005EE 799	Chery Tiggo 7 Pro белый	8
6	6	F006FF 799	Nissan Qashqai черный	9
7	7	G007GG 799	Changan UNI-5 серый	10
8	8	H008HH 799	BMW 318i синий	12
9	9	I009II 799	Audi A3 белый	13
10	10	J010JJ 799	Exeed TXL черный	14
11	11	K011KK 799	Tesla Model Y белый	16
12	13	M013MM 799	ГАЗ Газель белый	20
13	14	N014NN 799	УАЗ 3303 зеленый	21
14	15	O015OO 799	Sollers TR80 белый	23
15	16	P016PP 799	DONGFENG EV400 синий	24

Рисунок 10 – Таблица cars

В таблицу ei (единица измерения) данные были добавлены вручную в БД. Представлено на рисунке 10.

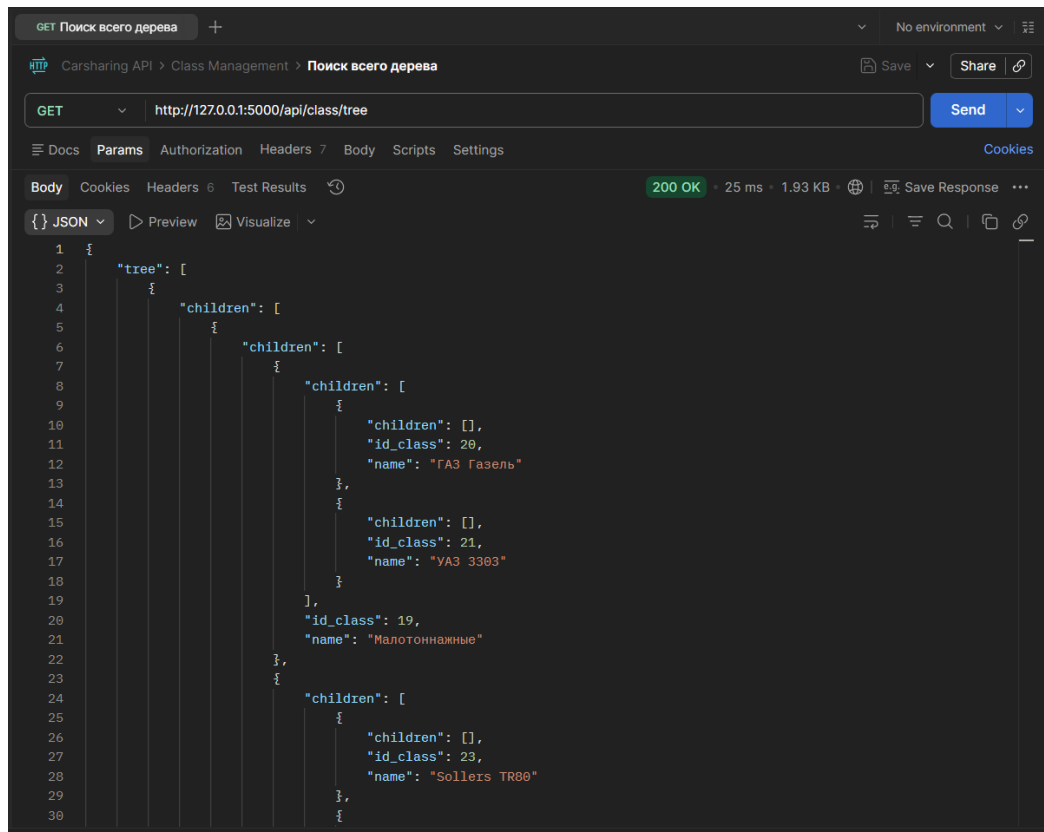
Database Structure Browse Data Edit Pragmas Execute SQL

Таблица: ei

	<u>id_ei</u>	short_name	name
	Фил...	Фильтр	Фильтр
1	1	шт	штуки

Рисунок 11 – Таблица ei

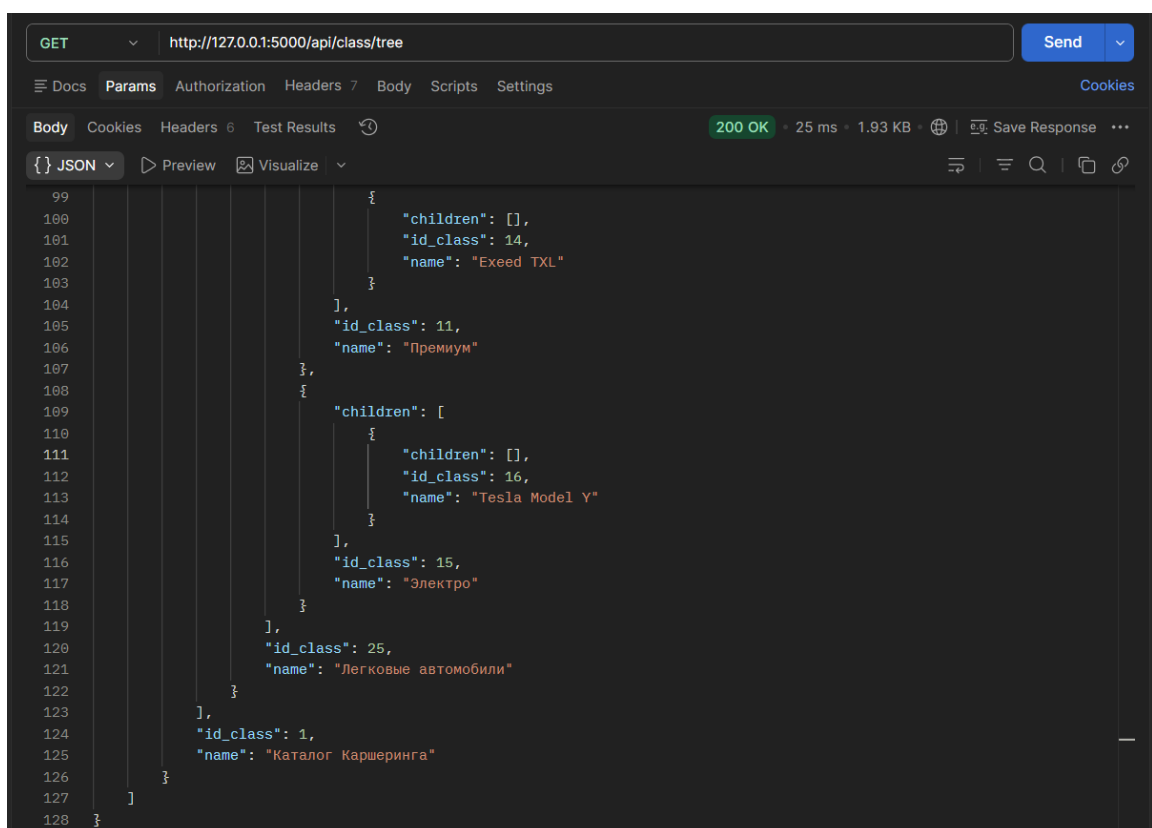
Протестируем метод вывода всего дерева. Результат работы метода представлен на рисунке 12 и 13.



The screenshot shows a REST client interface with the URL `http://127.0.0.1:5000/api/class/tree` and a GET request. The response is a JSON object with a `"tree"` property containing an array of objects. The visible part of the response is as follows:

```
1 {
2   "tree": [
3     {
4       "children": [
5         {
6           "children": [
7             {
8               "children": [
9                 {
10                  "children": [],
11                  "id_class": 20,
12                  "name": "ГАЗ Газель"
13                },
14                {
15                  "children": [],
16                  "id_class": 21,
17                  "name": "УАЗ 3303"
18                }
19              ],
20              "id_class": 19,
21              "name": "Малотоннажные"
22            },
23            {
24              "children": [
25                {
26                  "children": [],
27                  "id_class": 23,
28                  "name": "Sollers TR80"
29                }
30              ]
31            }
32          ]
33        }
34      ]
35    }
36  ]
37}
```

Рисунок 12 – Начало дерева



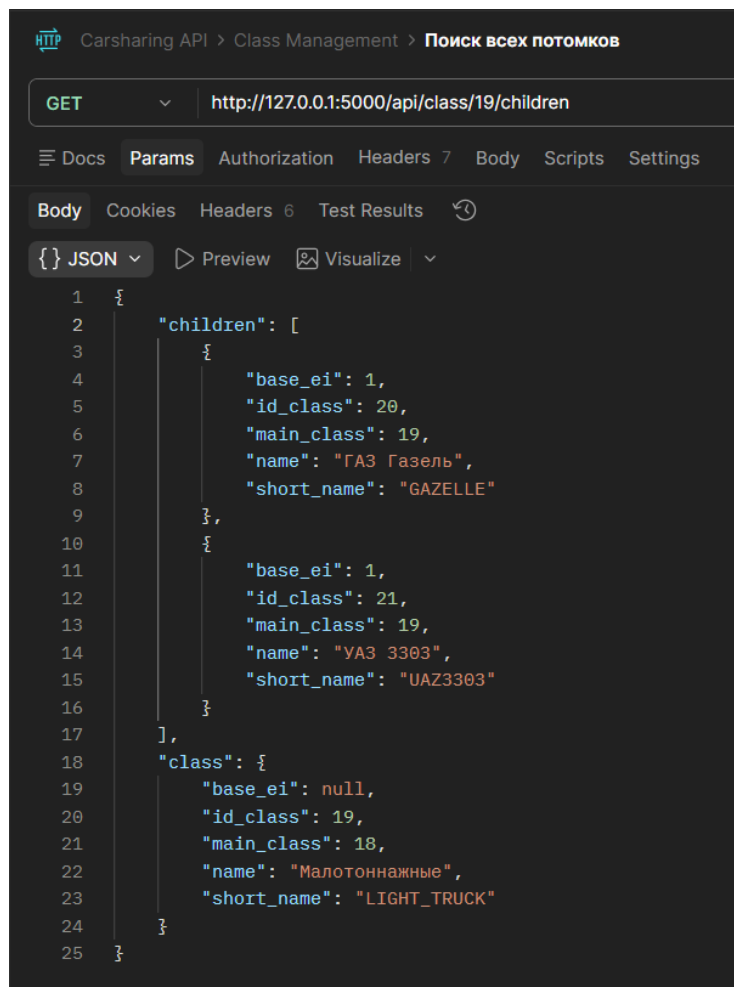
The screenshot shows the continuation of the JSON response from the previous image. The visible part of the response is as follows:

```
99     {
100       "children": [],
101       "id_class": 14,
102       "name": "Exeed TXL"
103     },
104     {
105       "id_class": 11,
106       "name": "Премиум"
107     },
108     {
109       "children": [
110         {
111           "children": [],
112           "id_class": 16,
113           "name": "Tesla Model Y"
114         },
115         {
116           "id_class": 15,
117           "name": "Электро"
118         }
119       ],
120       "id_class": 25,
121       "name": "Легковые автомобили"
122     },
123     {
124       "id_class": 1,
125       "name": "Каталог Каршеринга"
126     }
127   ]
128 }
```

Рисунок 13 – Конец дерева

Протестируем метод вывода всех потомков класса «Малотоннажные».

Результат работы метода представлен на рисунке 14.



```
1  {
2    "children": [
3      {
4        "base_ei": 1,
5        "id_class": 20,
6        "main_class": 19,
7        "name": "ГАЗ Газель",
8        "short_name": "GAZELLE"
9      },
10     {
11       "base_ei": 1,
12       "id_class": 21,
13       "main_class": 19,
14       "name": "УАЗ 3303",
15       "short_name": "UAZ3303"
16     }
17   ],
18   "class": {
19     "base_ei": null,
20     "id_class": 19,
21     "main_class": 18,
22     "name": "Малотоннажные",
23     "short_name": "LIGHT_TRUCK"
24   }
25 }
```

Рисунок 14 – Потомки класса «Малотоннажные»

Протестируем метод вывода всех родителей класса «Малотоннажные».

Результат работы метода представлен на рисунке 15.

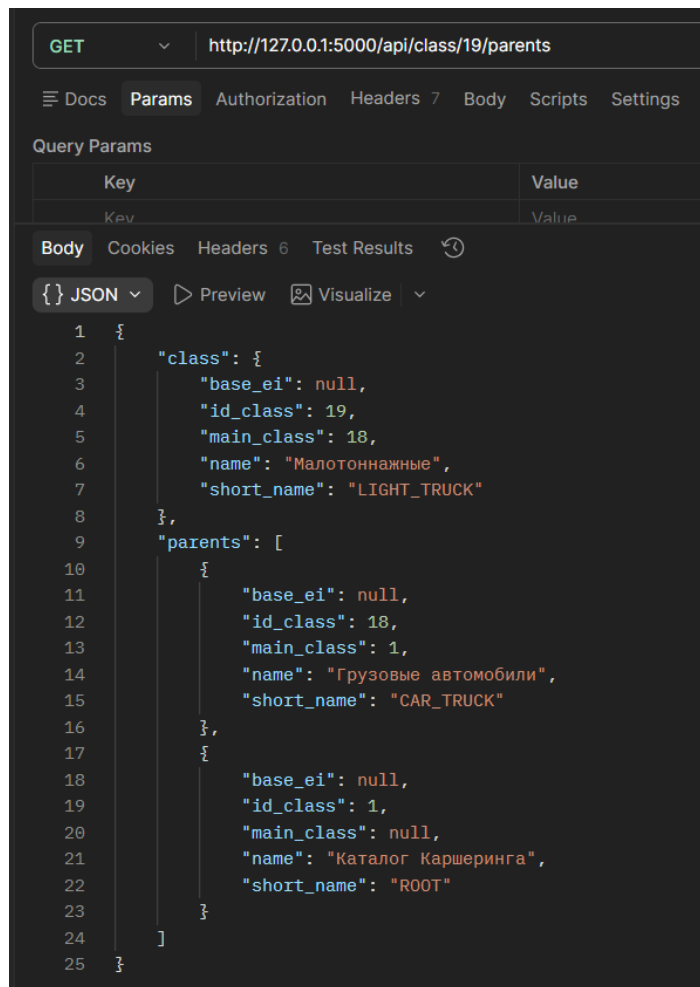


Рисунок 15 – Родители класса «Малотоннажные»

Протестируем метод удаление класса. Для удаление был выбран класс «Электро». Результат работы метода представлен на рисунке 16.

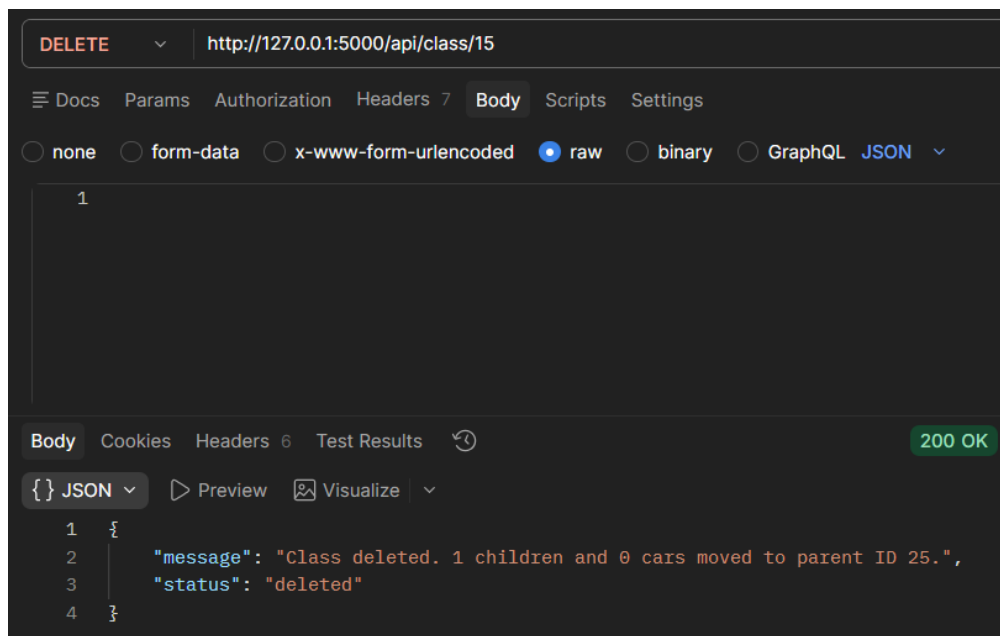


Рисунок 16 – Удаление класса «Электро»

Класс был успешно удален. Удаляя класс, все его потомки каскадно поднимаются показано на рисунке 17.

0	children	id_class	name	25	Легковые автомобили
0	children	id_class	name	3	Эконом
0	0	5	VW Polo VI		
0	1	6	Citroen C4		
0	2	26	Kia Rio		
1	children	id_class	name	7	Комфорт
1	0	8	Chery Tiggo 7 Pro		
1	1	9	Nissan Qashqai		
1	2	10	Changan UNI-5		
2	children	id_class	name	11	Премиум
2	0	12	BMW 318i		
2	1	13	Audi A3		
2	2	14	Exeed TXL		
3	children	id_class	name	16	Tesla Model Y

Рисунок 17 – Потомки класса «Легковые машины»

Протестируем метод перемещение класса. Переместим класс «Tesla Model Y» внутрь класса «Премиум». Результат работы метода представлен на рисунке 18.

Carsharing API > Class Management > Перемещение класса

PUT

http://127.0.0.1:5000/api/class/16/move

Docs

Params

Authorization

Headers 9

Body

Scripts

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1 {

2 | "new_parent_id": 11

3 }

Body

Cookies

Headers 6

Test Results

200 OK

{ } JSON

Preview

Visualize

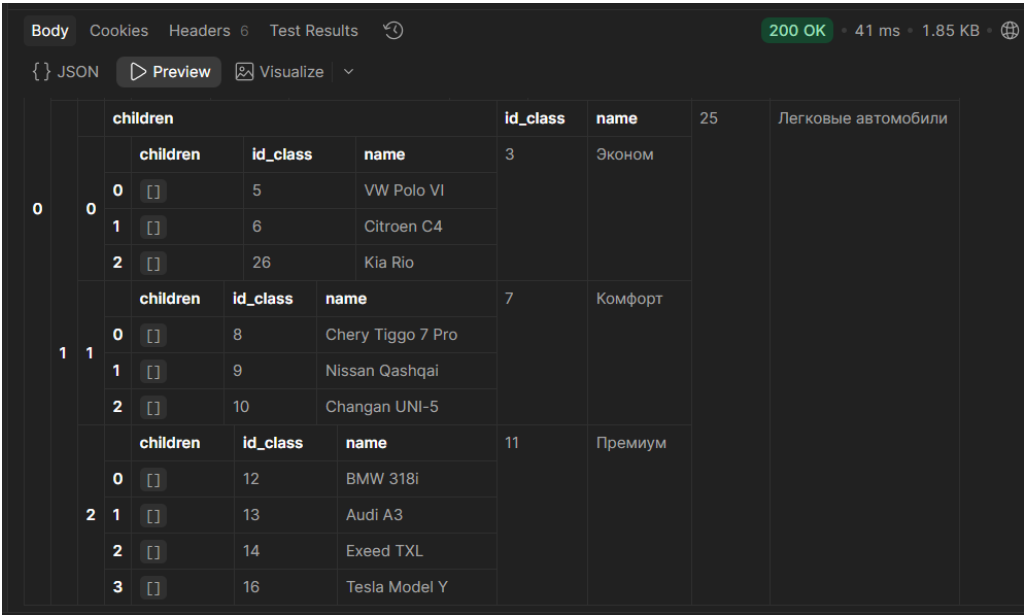
1 {

2 | "status": "moved"

3 }

Рисунок 18 – Перемещение класс «Премиум»

После успешного выполнения метода перемещение класса, проверяем вывод дерева показано на рисунке 19. Класс успешно перемещён.

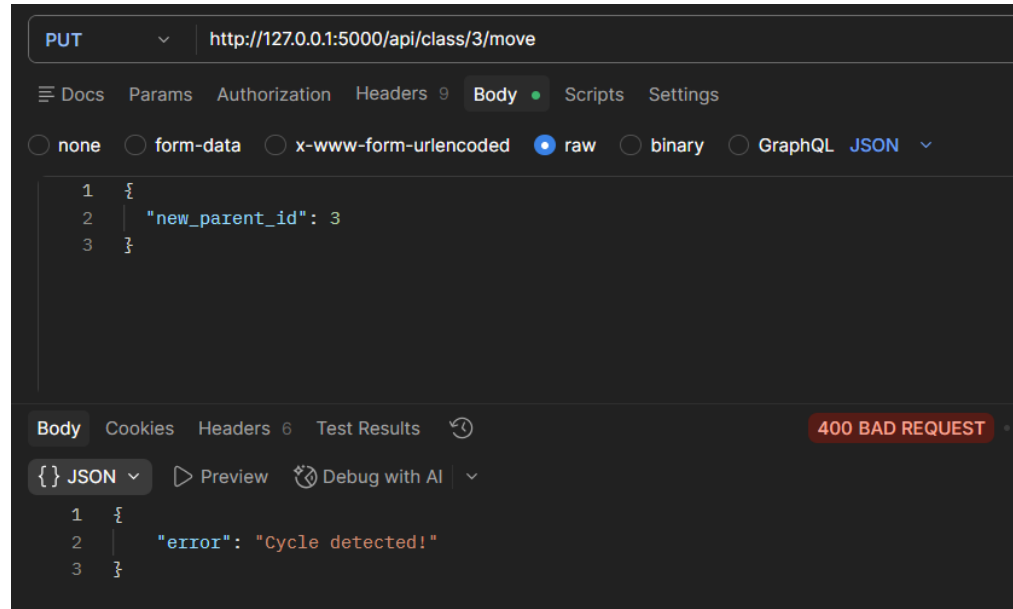


The screenshot shows a web browser interface with a JSON tree view. The tree is rooted at a node with 'id_class' 25 and 'name' 'Легковые автомобили'. It branches into three categories: 'Эконом' (id_class 3), 'Комфорт' (id_class 7), and 'Премиум' (id_class 11). Each category contains a list of car models with their respective 'id_class' and 'name'.

id_class	name
25	Легковые автомобили
3	Эконом
5	VW Polo VI
6	Citroen C4
26	Kia Rio
7	Комфорт
8	Chery Tiggo 7 Pro
9	Nissan Qashqai
10	Changan UNI-5
11	Премиум
12	BMW 318i
13	Audi A3
14	Exeed TXL
16	Tesla Model Y

Рисунок 19 – Потомки класса «Легковые машины»

Пробуем сделать класс своим собственным родителем (проверка на циклы). Результат теста успешен – вышла ошибка, цикл не образовался. Представлен на рисунке 20.



The screenshot shows a REST client interface. The top part displays a PUT request to 'http://127.0.0.1:5000/api/class/3/move' with a JSON body: { "new_parent_id": 3 }. The bottom part shows the response: 400 BAD REQUEST with a JSON body: { "error": "Cycle detected!" }.

```
PUT http://127.0.0.1:5000/api/class/3/move
{
  "new_parent_id": 3
}

400 BAD REQUEST
{
  "error": "Cycle detected!"
}
```

Рисунок 20 – Проверка на циклы

Протестируем метод вывода всех экземпляров класс (поиск всех машин). Результат метода представлен на рисунке 21 и 22.

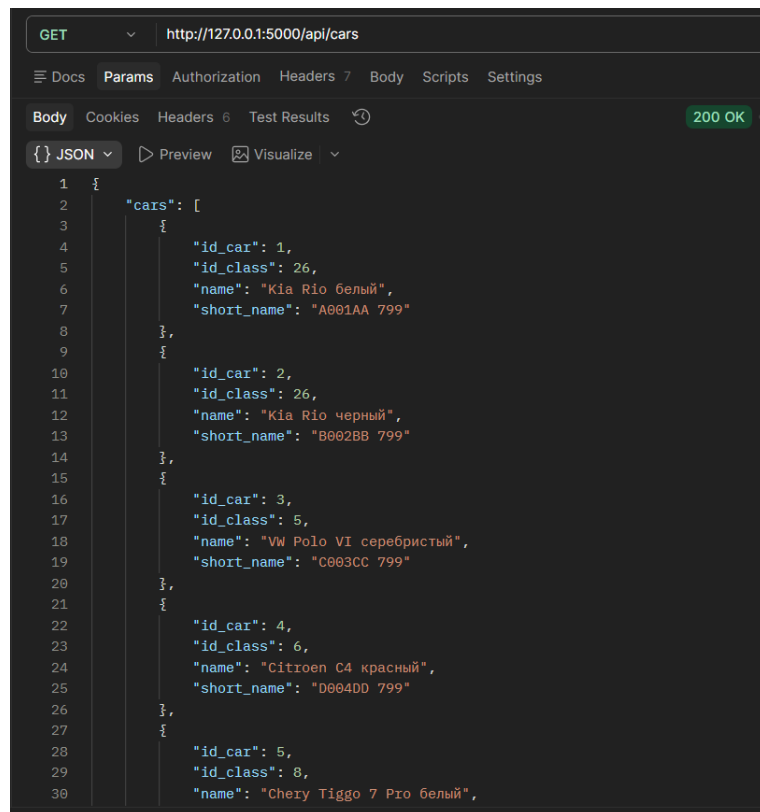


Рисунок 21 – Начало вывода всех экземпляров

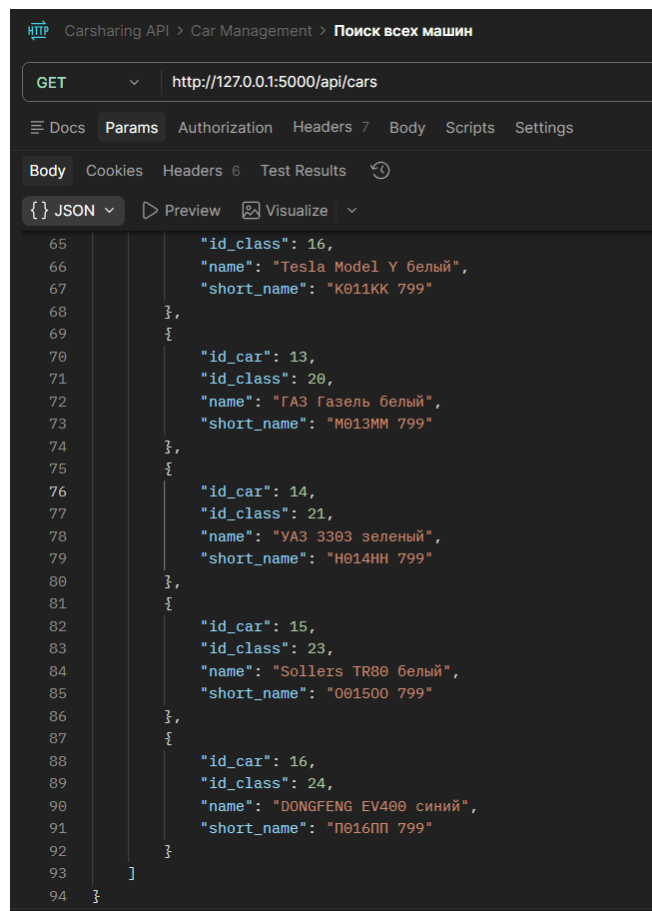


Рисунок 22 – Конец вывода экземпляров класса

Протестируем метод поиска экземпляров класса «Эконом». На рисунке 23 видим вывод всех экземпляров данного метода.

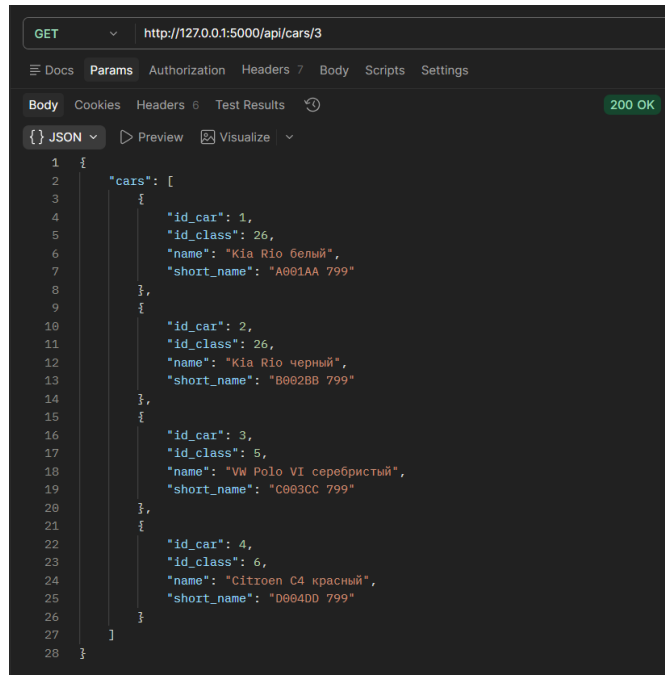


Рисунок 23 – Экземпляры класса «Эконом»

Протестируем метод работы удаление экземпляра класса. На рисунке 24 представлен результат работы метода удаление экземпляра.

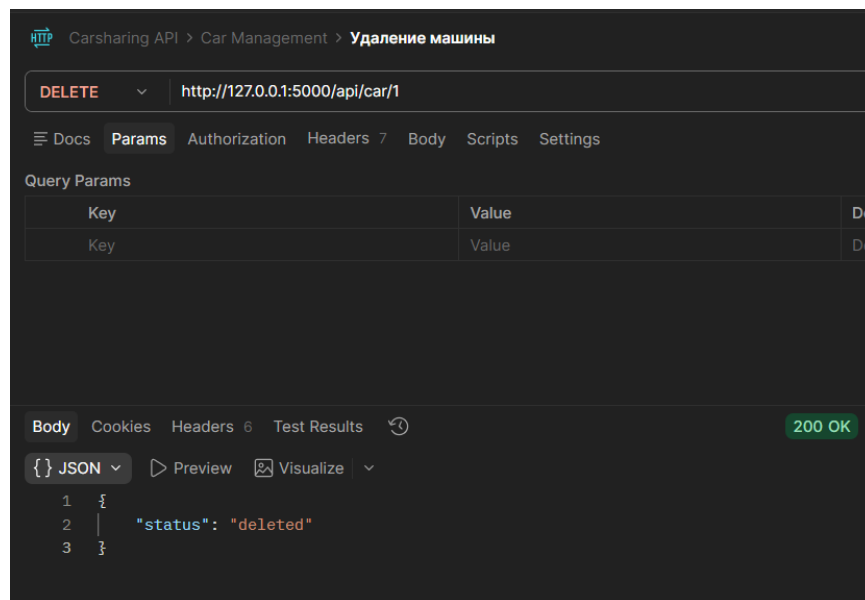


Рисунок 24 – Удаление машины с id = 1

Проверим удаление экземпляра класса выводом всех экземпляров класса «Эконом». На рисунке 25 мы видим, что экземпляр с id = 1 был успешно удален.

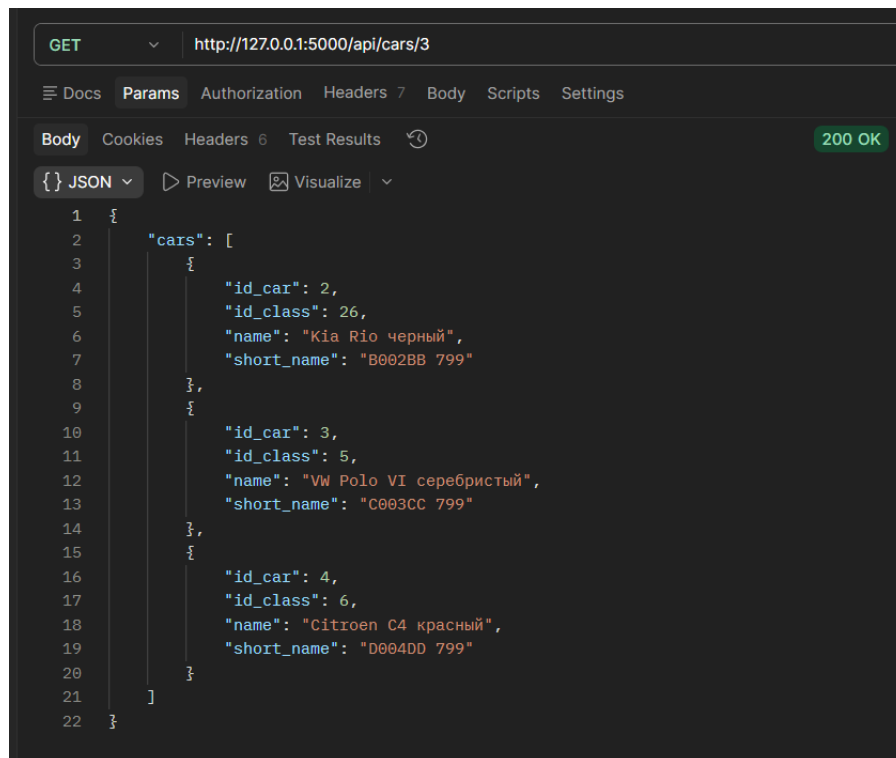


Рисунок 25 – Экземпляры класс «Эконом»

Протестируем метод перемещение экземпляра класса в другой класс. На рисунке 26 продемонстрировано успешное перемещение класса.

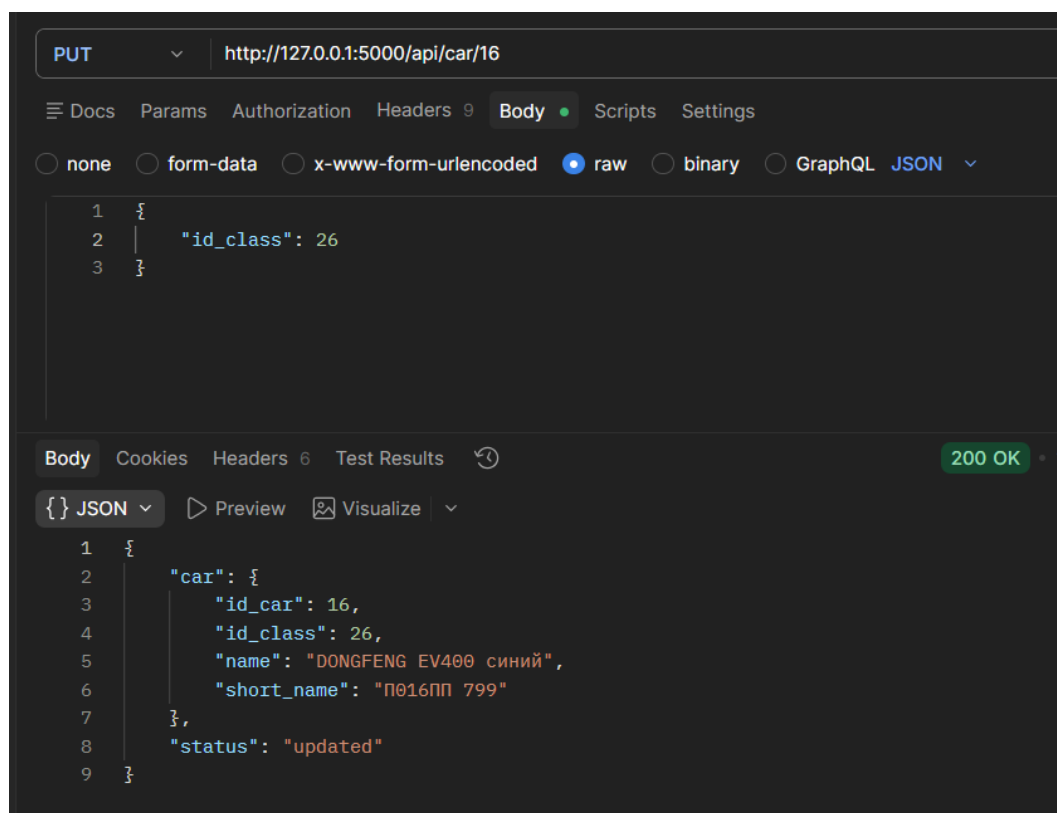
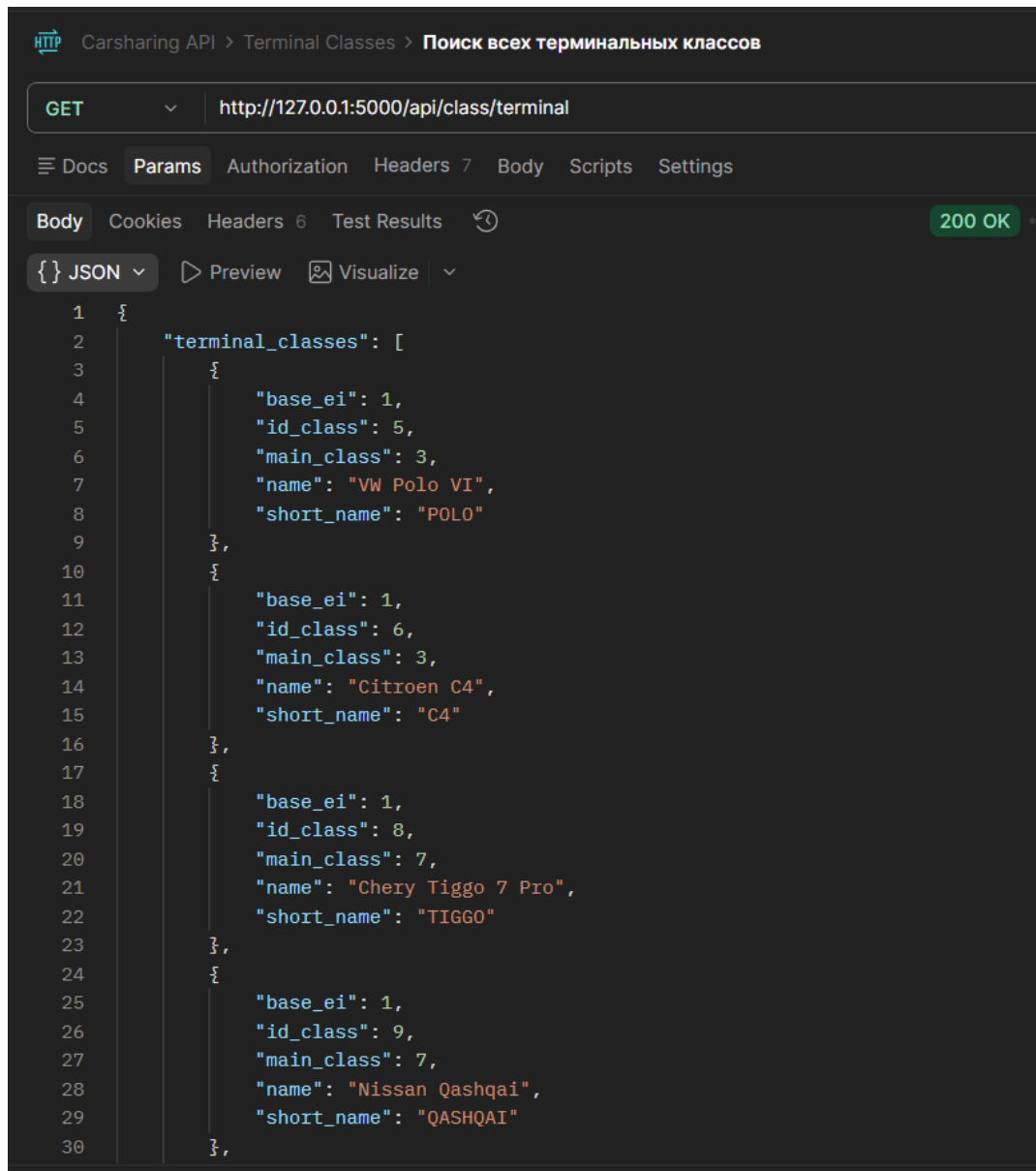


Рисунок 26 – Перемещение экземпляра класса

Протестируем метод вывода всех терминальных классов. Результат работы представлен на рисунках 27 и 28.



The screenshot shows a web browser interface for testing an API. The address bar displays the URL `http://127.0.0.1:5000/api/class/terminal`. The response status is `200 OK`. The response body is displayed in JSON format, showing a list of terminal classes.

```
1  {
2    "terminal_classes": [
3      {
4        "base_ei": 1,
5        "id_class": 5,
6        "main_class": 3,
7        "name": "VW Polo VI",
8        "short_name": "POLO"
9      },
10     {
11       "base_ei": 1,
12       "id_class": 6,
13       "main_class": 3,
14       "name": "Citroen C4",
15       "short_name": "C4"
16     },
17     {
18       "base_ei": 1,
19       "id_class": 8,
20       "main_class": 7,
21       "name": "Chery Tiggo 7 Pro",
22       "short_name": "TIGGO"
23     },
24     {
25       "base_ei": 1,
26       "id_class": 9,
27       "main_class": 7,
28       "name": "Nissan Qashqai",
29       "short_name": "QASHQAI"
30     }
31   ]
32 }
```

Рисунок 27 – Вывод всех терминальных классов (начало)

The screenshot shows a web browser interface for a REST client. The address bar displays the URL `http://127.0.0.1:5000/api/class/terminal`. The method is set to `GET`. The response status is `200 OK`. The response body is displayed in JSON format, showing an array of four terminal class objects. The JSON is as follows:

```
{
  "base_ei": 1,
  "id_class": 21,
  "main_class": 19,
  "name": "YA3 3303",
  "short_name": "UAZ3303"
},
{
  "base_ei": 1,
  "id_class": 23,
  "main_class": 22,
  "name": "Sollers TR80",
  "short_name": "SOLLERS"
},
{
  "base_ei": 1,
  "id_class": 24,
  "main_class": 22,
  "name": "DONGFENG EV400",
  "short_name": "DF_EV400"
},
{
  "base_ei": 1,
  "id_class": 26,
  "main_class": 3,
  "name": "Kia Rio",
  "short_name": "RIO"
}
```

Рисунок 28 – Вывод всех термальных классов (конец)

Таким образом все методы были успешно протестированы, что показывается корректность реализации БД и серверной части.

ЗАКЛЮЧЕНИЕ

В ходе выполнения практической работы был разработан полнофункциональный прототип системы управления иерархическим классификатором автомобилей для платформы каршеринга. Цель работы была достигнута.

Разработана иерархическая модель данных для классификатора автомобилей каршеринга, поддерживающая вложенность категорий. Спроектирована реляционная база данных на SQLite с использованием ORM SQLAlchemy, включающая таблицы для узлов классификатора. Реализован REST API сервер на Flask, предоставляющий полный набор методов для управления структурой дерева и экземплярами. Обеспечена целостность данных: внедрены алгоритмы проверки на циклические зависимости при изменении иерархии и реализовано каскадное переподчинение дочерних элементов при удалении родительских узлов. Проведено успешное тестирование системы: подтверждена корректная работа всех эндпоинтов API.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Ситидрайв – каршеринг в Санкт-Петербурге // CityDrive. URL: <https://citydrive.ru/spb/carsharing/> (дата обращения: 24.05.2026).
2. Радзишевский А. Проектирование Информационных систем. Часть 1. Введение // Хабр. 2025. URL: <https://habr.com/ru/articles/912914/> (дата обращения: 24.05.2026).
3. Дубенецкий В. А., Кузнецов А. Г. Использование объектных моделей при создании информационных систем: учеб.-метод. пособие. СПб.: Изд-во СПбГЭТУ «ЛЭТИ», 2016. 220 с.
4. Дубенецкий В. А., Кузнецов А. Г. Проектирование информационных систем с использованием UML: учеб.-метод. пособие. СПб.: Изд-во СПбГЭТУ «ЛЭТИ», 2016. 62 с.

ПРИЛОЖЕНИЕ А. ЛИСТИНГ

```
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy
from flask_cors import CORS

app = Flask(__name__)
CORS(app)  # Разрешить запросы из браузера

# Настройка базы данных (SQLite)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///database.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
db = SQLAlchemy(app)

# МОДЕЛИ

class CarClass(db.Model):
    """Метакласс для иерархии классификатора (таблица
    car_classes)"""
    __tablename__ = 'car_classes'

    id_class = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(100), nullable=False)
    short_name = db.Column(db.String(50))
    base_ei = db.Column(db.Integer,
db.ForeignKey('ei.id_ei'))  # Ссылка на таблицу ei
    main_class = db.Column(db.Integer,
db.ForeignKey('car_classes.id_class'))  # Рефлексивная связь

    # Рефлексивная связь (родитель → потомки)
    parent = db.relationship('CarClass',
                             backref=db.backref('children',
lazy='dynamic'),
                             remote_side=[id_class])

    def to_dict(self):
        return {
            'id_class': self.id_class,
            'name': self.name,
            'short_name': self.short_name,
            'base_ei': self.base_ei,
            'main_class': self.main_class
        }

    def check_cycle(self, new_parent_id):
        """Проверка на циклы при смене родителя"""
        if new_parent_id == self.id_class:
            return True
        current = CarClass.query.get(new_parent_id)
        while current:
            if current.id_class == self.id_class:
                return True
```

```

        current = current.parent
    return False

def get_all_children(self):
    """Рекурсивно найти всех потомков"""
    result = list(self.children)
    for child in self.children:
        result.extend(child.get_all_children())
    return result

def get_all_parents(self):
    """Найти всех предков"""
    result = []
    current = self.parent
    while current:
        result.append(current)
        current = current.parent
    return result

class Car(db.Model):
    "Конкретные автомобили (таблица cars)"
    __tablename__ = 'cars'

    id_car = db.Column(db.Integer, primary_key=True)
    short_name = db.Column(db.String(20), unique=True,
        nullable=False) # Госномер
    name = db.Column(db.String(100)) # Описание (напр. Kia Rio
        белый)
    id_class = db.Column(db.Integer,
        db.ForeignKey('car_classes.id_class'), nullable=False)

    car_class = db.relationship('CarClass', backref='cars')

    def to_dict(self):
        return {
            'id_car': self.id_car,
            'short_name': self.short_name,
            'name': self.name,
            'id_class': self.id_class
        }

class Unit(db.Model):
    "Единицы измерения (таблица ei)"
    __tablename__ = 'ei'

    id_ei = db.Column(db.Integer, primary_key=True)
    short_name = db.Column(db.String(10), unique=True,
        nullable=False) # Код (шт, км)
    name = db.Column(db.String(50), nullable=False) # Название
        (штуки, километры)

    def to_dict(self):

```

```

        return {'id_ei': self.id_ei, 'short_name':
self.short_name, 'name': self.name}

# API ЭНДПОИНТЫ (Методы сервера)

@app.route('/api/class/add', methods=['POST'])
def add_class():
    """Добавить новый класс (вершину)"""
    data = request.json
    main_class = data.get('main_class')

    # Проверка на цикл (если указан родитель)
    if main_class:
        # Временная проверка (упрощенная)
        pass

    new_class = CarClass(
        name=data['name'],
        short_name=data.get('short_name', ''),
        base_ei=data.get('base_ei'),
        main_class=main_class
    )
    db.session.add(new_class)
    db.session.commit()
    return jsonify({'status': 'created', 'id_class':
new_class.id_class}), 201

@app.route('/api/class/<int:id_class>', methods=['DELETE'])
def delete_class(id_class):
    clazz = CarClass.query.get(id_class)
    if not clazz:
        return jsonify({'error': 'Class not found'}), 404

    new_parent_id = clazz.main_class
    cars_in_class = Car.query.filter_by(id_class=id_class).all()

    # Если у класса нет родителя
    if new_parent_id is None:
        if len(cars_in_class) > 0 or clazz.children.count() > 0:
            return jsonify({
                'error': 'Cannot delete root class that contains
children or cars.'
            }), 400

    db.session.delete(clazz)
    db.session.commit()
    return jsonify({'status': 'deleted'}), 200

    # Переносим детей
    children =
CarClass.query.filter_by(main_class=id_class).all()
    for child in children:
        child.main_class = new_parent_id

```

```

# Переносим машины
for car in cars_in_class:
    car.id_class = new_parent_id

# Фиксируем изменения перед удалением
db.session.flush()

# Удаляем класс
db.session.delete(clazz)

try:
    db.session.commit()
    return jsonify({
        'status': 'deleted',
        'message': f'Class deleted. {len(children)} children
and {len(cars_in_class)} cars moved to parent ID
{new_parent_id}.'
    }), 200
except Exception as e:
    db.session.rollback()
    return jsonify({'error': str(e)}), 500

@app.route('/api/class/<int:id_class>/move', methods=['PUT'])
def move_class(id_class):
    "Переместить класс (сменить родителя)"
    data = request.json
    new_parent_id = data.get('new_parent_id')

    clazz = CarClass.query.get(id_class)
    if not clazz:
        return jsonify({'error': 'Class not found'}), 404

    # Проверка на циклы
    if clazz.check_cycle(new_parent_id):
        return jsonify({'error': 'Cycle detected!'}), 400

    clazz.main_class = new_parent_id
    db.session.commit()
    return jsonify({'status': 'moved'}), 200

@app.route('/api/class/<int:id_class>/children',
methods=['GET'])
def get_children(id_class):
    "Найти всех потомков класса"
    clazz = CarClass.query.get(id_class)
    if not clazz:
        return jsonify({'error': 'Class not found'}), 404

    all_children = clazz.get_all_children()
    return jsonify({
        'class': clazz.to_dict(),
        'children': [c.to_dict() for c in all_children]
    })

```

```

   )), 200

@app.route('/api/class/<int:id_class>/parents', methods=['GET'])
def get_parents(id_class):
    "Найти всех родителей (предков) класса"
    clazz = CarClass.query.get(id_class)
    if not clazz:
        return jsonify({'error': 'Class not found'}), 404

    all_parents = clazz.get_all_parents()
    return jsonify({
        'class': clazz.to_dict(),
        'parents': [p.to_dict() for p in all_parents]
    }), 200

@app.route('/api/class/terminal', methods=['GET'])
def get_terminal_classes():
    "Найти все терминальные классы (листья дерева)"
    all_classes = CarClass.query.all()
    terminal = [c for c in all_classes if c.children.count() ==
0]
    return jsonify({'terminal_classes': [c.to_dict() for c in
terminal]}), 200

@app.route('/api/class/tree', methods=['GET'])
def get_tree():
    "Просмотреть всю структуру классификатора"
    roots = CarClass.query.filter_by(main_class=None).all()

    def build_tree(node):
        return {
            'id_class': node.id_class,
            'name': node.name,
            'children': [build_tree(child) for child in
node.children]
        }

    return jsonify({'tree': [build_tree(r) for r in roots]}),
200

@app.route('/api/car/add', methods=['POST'])
def add_car():
    "Добавить конкретный автомобиль"
    data = request.json
    new_car = Car(
        short_name=data['short_name'], # Госномер
        name=data.get('name'),
        id_class=data['id_class']
    )
    db.session.add(new_car)
    db.session.commit()
    return jsonify({'status': 'created', 'id_car':
new_car.id_car}), 201

```

```

@app.route('/api/car/<int:id_car>', methods=['DELETE'])
def delete_car(id_car):
    "Удалить конкретный автомобиль"
    car = Car.query.get(id_car)
    if not car:
        return jsonify({'error': 'Car not found'}), 404

    db.session.delete(car)
    db.session.commit()
    return jsonify({'status': 'deleted'}), 200

@app.route('/api/car/<int:id_car>', methods=['PUT'])
def update_car(id_car):
    "Обновить данные автомобиля"
    car = Car.query.get(id_car)
    if not car:
        return jsonify({'error': 'Car not found'}), 404

    data = request.json

    # Обновляем класс, если он передан
    if 'id_class' in data:
        new_class_id = data['id_class']

        # ПРОВЕРКА СУЩЕСТВОВАНИЯ ЦЕЛЕВОГО КЛАССА
        target_class = CarClass.query.get(new_class_id)
        if not target_class:
            return jsonify({'error': f'Target class with id {new_class_id} does not exist'}), 404

        car.id_class = new_class_id

    # Обновляем описание/название, если передано
    if 'name' in data:
        car.name = data['name']

    try:
        db.session.commit()
        return jsonify({'status': 'updated', 'car': car.to_dict()}), 200
    except Exception as e:
        db.session.rollback()
        return jsonify({'error': str(e)}), 500

@app.route('/api/cars', methods=['GET'])
def get_cars():
    "Получить список всех автомобилей"
    cars = Car.query.all()
    return jsonify({'cars': [c.to_dict() for c in cars]}), 200

@app.route('/api/cars/<int:id_class>', methods=['GET'])

```

```

def get_cars_by_class(id_class):
    "Получить автомобили по классу (включая потомков)"
    clazz = CarClass.query.get(id_class)
    if not clazz:
        return jsonify({'error': 'Class not found'}), 404

    # Получить все потомков (включая сам класс)
    all_classes = [clazz] + clazz.get_all_children()
    class_ids = [c.id_class for c in all_classes]

    cars = Car.query.filter(Car.id_class.in_(class_ids)).all()
    return jsonify({'cars': [c.to_dict() for c in cars]}), 200

# ЗАПУСК СЕРВЕРА

if __name__ == '__main__':
    with app.app_context():
        db.create_all() # Создать таблицы БД

        # Добавить корневой класс и единицу измерения, если нет
        if not Unit.query.first():
            db.session.add(Unit(short_name='шт', name='штуки'))
            db.session.commit()

        if not CarClass.query.first():
            root = CarClass(name='Каталог Каршеринга',
short_name='ROOT')
            db.session.add(root)
            db.session.commit()
            print('База данных создана и заполнена тестовыми
данными!')

    print('Сервер запущен на http://127.0.0.1:5000')
    app.run(debug=False, port=5000)

```